

[GSC]1-yolov7 训练验证

- [环境信息](#)
- [基础信息](#)
- [模型数据集信息](#)
- [参看文档](#)
- [环境准备](#)
- [驱动安装](#)
- [Docker 加载](#)
 - [docker images 查看当前系统加载镜像](#)
 - [docker load 镜像](#)
- [进入docker 测试容器](#)
- [代码适配](#)
- [代码和模型下载](#)
- [代码适配 \(MLU\)](#)
 - [修改说明](#)
 - [代码修改](#)
 - [使用torch_gpu2mlu工具进行修改](#)
 - [其它代码修改](#)
- [验证测试 \(MLU\)](#)
 - [安装依赖](#)
 - [数据集准备](#)
 - [训练验证](#)
 - [推理验证](#)

环境信息

基础信息

模块	信息	备注
板卡	MLU370-X8	
SDK版本	Pytorch1.9	
Docker镜像	yellow.hub.cambricon.com/pytorch/pytorch:v1.14.0-torch1.9-ubuntu18.04-py37	

模型数据集信息

模块	信息	备注
代码	https://github.com/WongKinYiu/yolov7.git	master
模型	https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-w6.pt	
数据集	COC02017	

参看文档

- [寒武纪PyTorch v1.9网络移植手册](#)
- [寒武纪PyTorch v1.9 API 支持列表](#)
- [寒武纪驱动安装包使用手册](#)
- [寒武纪 CNMon 用户手册](#)

环境准备

驱动安装

- 测试环境需要安装MLU驱动，可以通过 `**cnmon **` 检查当前环境情况以及MLU办卡信息等

Wed Jun 14 11:19:10 2023

CNMON v5.10.10				Driver v5.10.10			
Card	VF	Name	Firmware	Bus-Id	Util	Ecc-Error	
Fan	Temp		Pwr:Usage/Cap	Memory-Usage	SR-IOV	Compute-Mode	
0	/	MLU370-X8	v1.1.6	0000:24:00.0	0%	N/A	
0%	32C		104 W/ 250 W	0 MiB/ 23374 MiB	N/A	Default	
1	/	MLU370-X8	v1.1.6	0000:27:00.0	0%	N/A	
0%	39C		104 W/ 250 W	0 MiB/ 23374 MiB	N/A	Default	
2	/	MLU370-X8	v1.1.6	0000:2D:00.0	0%	N/A	
0%	31C		104 W/ 250 W	0 MiB/ 23374 MiB	N/A	Default	
3	/	MLU370-X8	v1.1.6	0000:30:00.0	0%	N/A	
0%	39C		104 W/ 250 W	0 MiB/ 23374 MiB	N/A	Default	
Processes:							
Card	MI	PID	Command Line			MLU Memory Usage	
No running processes found							

Docker 加载

- 所有示例均基于docker运行，运行测试例子前请检查docker 是否加载

docker images 查看当前系统加载镜像

yellow.hub.cambricon.com/pytorch/pytorch
4fec082a9bc1 3 weeks ago 8.84GB

v1.14.0-torch1.9-ubuntu18.04-py37

docker load 镜像

- 如果未加载镜像，可使用docker load进行加载

sudo docker load -i pytorch-v1.14.0-torch1.9-ubuntu18.04-py37.tar.gz

进入docker 测试容器

- 启动容器脚本参看 (run_pytorch1.9.sh)

```
#!/bin/bash

ver=v1.14.0
export MY_CONTAINER="mlu370_pytorch_ubuntu18.04_torch1.9_guoshoucai_${ver}_`whoami`"
export IMAGE_NAME="yellow.hub.cambricon.com/pytorch/pytorch:${ver}-torch1.9-ubuntu18.04-py37"

num=`docker ps -a|grep -w "$MY_CONTAINER"|wc -l`
echo $num
echo $MY_CONTAINER
echo $IMAGE_NAME

if [ 0 -eq $num ];then
    xhost +
    docker run \
        -e DISPLAY=unix$DISPLAY \
        --net=host \
        --shm-size '64gb' \
        -it --privileged \
        --name $MY_CONTAINER \
        -v /data:/data/ \
        -v /opt/share/datasets:/datasets/\
        -v /opt/share/models:/models/\
        -v $PWD:/workspace/ \
        -w /workspace/\
        -v /usr/bin/cnmon:/usr/bin/cnmon \
        $IMAGE_NAME \
        /bin/bash
else
    docker start $MY_CONTAINER
    docker exec -ti $MY_CONTAINER /bin/bash
fi
```

注意：后续测试所有操作，均在容器内进行。

代码适配

代码和模型下载

如果本地已经有代码和测试模型，忽略该步骤。

```
git clone https://github.com/WongKinYiu/yolov7.git
wget https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-w6.pt
```

代码适配 (MLU)

修改说明

原生代码通常可以基于CUDA或者是CPU运行，如果在基于MLU侧运行，需要对测试代码进行微调，主要调整点为：

1. 代码中对device的操作，如：`.cuda`修改为`.mlu`，修改规则 可以参看寒武纪官方文档：[寒武纪PyTorch v1.9网络移植手册](#)和[寒武纪PyTorch v1.9 API 支持列表文档](#)。

代码修改

使用torch_gpu2mlu工具进行修改

```
python /torch/src/catch/tools/torch_gpu2mlu.py -i yolov7/

# Cambricon PyTorch Model Migration Report
Official PyTorch model scripts: /home/share/jinrong/rezhou/guoshoucai/samples/1-yolov7/yolov7
Cambricon PyTorch model scripts: /home/share/jinrong/rezhou/guoshoucai/samples/1-yolov7/yolov7_mlu
Migration Report: /home/share/jinrong/rezhou/guoshoucai/samples/1-yolov7/yolov7_mlu/report.md
```

备注：修改后会生成带`**_mlu**` 目录，如`yolov7_mlu`，同时会生成代码修改记录等信息，详细参看[寒武纪PyTorch v1.9网络移植手册文档5. 使用工具迁移5.1. torch_gpu2mlu章节](#)。

其它代码修改

- 注释掉DP 代码如：`torch.nn.DataParallel`代码，多卡采用DDP方式(文件`train.py` 和 `train_aux.py`)

```
# DP mode
# if mlu and rank == -1 and torch.mlu.device_count() > 1:
#     model = torch.nn.DataParallel(model)
```

- 修改`utils/loss.py`中的`class ComputeLossAuxOTA`，将`gain`的初始化由`long`型设置为`int`型，即 `gain = torch.ones(7, device=targets.device).int()`。（有多处，都需要修改下）

```
#gain = torch.ones(7, device=targets.device).long() # normalized to gridspace gain
gain = torch.ones(7, device=targets.device).int() # normalized to gridspace gain
gain[2:6] = torch.tensor(p[i].shape)[3, 2, 3, 2]] # xyxy gain
```

备注：使用`long 0`，然后进行切片的位置需要进行修改

验证测试（MLU）

测试时需要确认当前运行环境信息，系统依赖、数据集、测试模型等

安装依赖

```
cd yolov7_mlu
#torch(MLUTORCH)
sed -i 's/torch/# torch/' requirements.txt
pip install -r requirements.txt
pip install pycocotools
```

数据集准备

官方默认使用coco数据集，默认代码数据集`data/coco.yaml` 描述，路径为当前目录`./coco/`下，如果客户使用自己数据集，按照实际进行路径调整。

```
cd yolov7_mlu
```

```
mkdir coco
ln -sf .
```

训练验证

测试命令同原生代码，注意单GPU和分布式训练差异。

```
# MLU
```

```
export MLU_VISIBLE_DEVICES=0,1,2,3
```

```
# train p5 models
```

```
python -m torch.distributed.launch --nproc_per_node 4 --master_port 9527 train.py --workers 8 --device 0,1,2,3 --sync-bn --batch-size 128 --data data/coco.yaml --img 640 640 --cfg cfg/training/yolov7.yaml --weights '' --name yolov7 --hyp data/hyp.scratch.p5.yaml
```

```
# train p6 models
```

```
python -m torch.distributed.launch --nproc_per_node 4 --master_port 9527 train_aux.py --workers 8 --device 0,1,2,3 --sync-bn --batch-size 32 --data data/coco.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6.yaml --weights '' --name yolov7-w6 --hyp data/hyp.scratch.p6.yaml
```

```
# MLU
```

```
#train p5 models
```

```
python train.py --workers 8 --device 0 --batch-size 32 --data data/coco.yaml --img 640 640 --cfg cfg/training/yolov7.yaml --weights '' --name yolov7 --hyp data/hyp.scratch.p5.yaml
```

```
# train p6 models
```

```
python train_aux.py --workers 8 --device 0 --batch-size 16 --data data/coco.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6.yaml --weights '' --name yolov7-w6 --hyp data/hyp.scratch.p6.yaml
```

Epoch	mlu_mem	box	obj	cls	total	labels	img_size
3/299	17G	0.04379	0.05926	0.03211	0.1352	93	1280: 100% 3697/3697 [2:02:02<00:00, 1.98s/it]
Class	Images	Labels		P	R	mAP@.5	mAP@.5:.95: 100% 313/313 [04:24<00:00, 1.19 it/s]
all	5000	36780	0.402	0.315	0.29	0.183	

备注:

- yolov7-w6 默认参数要30G内存，X8单芯24G，会出现申请失败
- X8是一卡双芯，可使用分布式将单卡完整利用起来。

推理验证

- 验证命令

```
python test.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.65 --device 0 --weights yolov7-w6.pt --name yolov7_640_val
```

- 测试结果

val: Scanning 'coco/val2017.cache' images and labels... 5000 found, 0 missing, 48 empty, 0 corrupted: 100%|| 5000/5000 [00:00<?, ?it/s]

Class	Images	Labels	P	R	mAP@.5	mAP@.5:.95:	0%
							0/157 [00:00<?, ?it/s]
[2023-6-14 14:47:34] [CNL] [Warning]:[cnlSetNmsDescriptor_v2] is deprecated and will be removed in the future release, please use [cnlSetNmsDescriptor_v5] instead.							
[2023-6-14 14:47:34] [CNL] [Warning]:[cnlClip] is deprecated and will be removed in the future release, please use [cnlClip_v2] instead.							

Class	Images	Labels	P	R	mAP@.5	mAP@.5:.95:	100% 157/157 [02:17<00:00, 1.15it/s]
all	5000	36780	0.724	0.622	0.671	0.485	

Speed: 6.6/2.6/9.2 ms inference/NMS/total per 640x640 image at batch-size 32

Evaluating pycocotools mAP... saving runs/test/yolov7_640_val/yolov7-w6_predictions.json...
loading annotations into memory...

Done (t=0.47s)
creating index...
index created!
Loading and preparing results...
DONE (t=3.23s)
creating index...
index created!

Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=59.71s).
Accumulating evaluation results...
DONE (t=9.44s).

Average Precision	(AP) @[IoU=0.50:0.95 area= all maxDets=100]	= 0.501
Average Precision	(AP) @[IoU=0.50 area= all maxDets=100]	= 0.680
Average Precision	(AP) @[IoU=0.75 area= all maxDets=100]	= 0.541
Average Precision	(AP) @[IoU=0.50:0.95 area= small maxDets=100]	= 0.301
Average Precision	(AP) @[IoU=0.50:0.95 area=medium maxDets=100]	= 0.553
Average Precision	(AP) @[IoU=0.50:0.95 area= large maxDets=100]	= 0.693
Average Recall	(AR) @[IoU=0.50:0.95 area= all maxDets= 1]	= 0.379
Average Recall	(AR) @[IoU=0.50:0.95 area= all maxDets= 10]	= 0.619
Average Recall	(AR) @[IoU=0.50:0.95 area= all maxDets=100]	= 0.668
Average Recall	(AR) @[IoU=0.50:0.95 area= small maxDets=100]	= 0.483
Average Recall	(AR) @[IoU=0.50:0.95 area=medium maxDets=100]	= 0.732
Average Recall	(AR) @[IoU=0.50:0.95 area= large maxDets=100]	= 0.843

Results saved to runs/test/yolov7_640_val